**Université Mundiapolis**
Campus Nouaceur
Filière : Ingénierie Informatique

# TP 02

Module : Blockchain

Preuve d'enjeu (Proof of Stake)

## 1 Principe et règles

En Proof of Stake, il n'y a pas de calcul coûteux comme en Proof of Work. Le protocole tire au sort le validateur qui produira le prochain bloc, et la probabilité d'être tiré est proportionnelle à la mise engagée (*stake*).

Si un validateur $i$ a engagé une mise $m_i$, sa probabilité d'être sélectionné vaut :

$$p_i = \frac{m_i}{\sum_j m_j}$$

Le TP consiste à vérifier cette formule par la simulation, sur un réseau que l'on construit soi-même.

Les règles ci-dessous fixent le protocole.

- **R1 Mise.** Un participant entre dans le réseau en bloquant une mise $m_i \geq 10$ ETH. La mise reste bloquée : elle sert de garantie.
- **R2 Sélection.** Avant chaque bloc, le réseau tire un nombre $t$ au hasard dans $[0\,;M]$, où $M = \sum_j m_j$. Le proposant est le validateur dont l'intervalle de mise contient $t$, d'où $p_i = m_i/M$.
- **R3 Un seul proposant.** Un bloc a exactement un proposant, celui que R2 désigne. Les autres validateurs n'écrivent rien.
- **R4 Tirage public.** Le bloc enregistre le nombre tiré $t$ et les mises du moment. N'importe quel participant peut donc refaire le calcul de R2 et retrouver le proposant attendu.
- **R5 Récompense.** Le proposant d'un bloc voit sa mise augmenter de $r \times m_i$, avec $r = 0{,}3\,\%$ par défaut. Les autres mises ne changent pas.
- **R6 Validité.** Un bloc est valide si le nom qu'il porte est celui que R4 permet de recalculer. Sinon il est falsifié.
- **R7 Sanction.** Un validateur qui a signé un bloc falsifié perd la moitié de sa mise et est exclu du réseau (*slashing*).

## 2 Mise en place

Le réseau est un serveur Flask hébergé par un nœud, auquel chaque participant se connecte. Aucune variable Python n'est partagée, tout passe par HTTP.

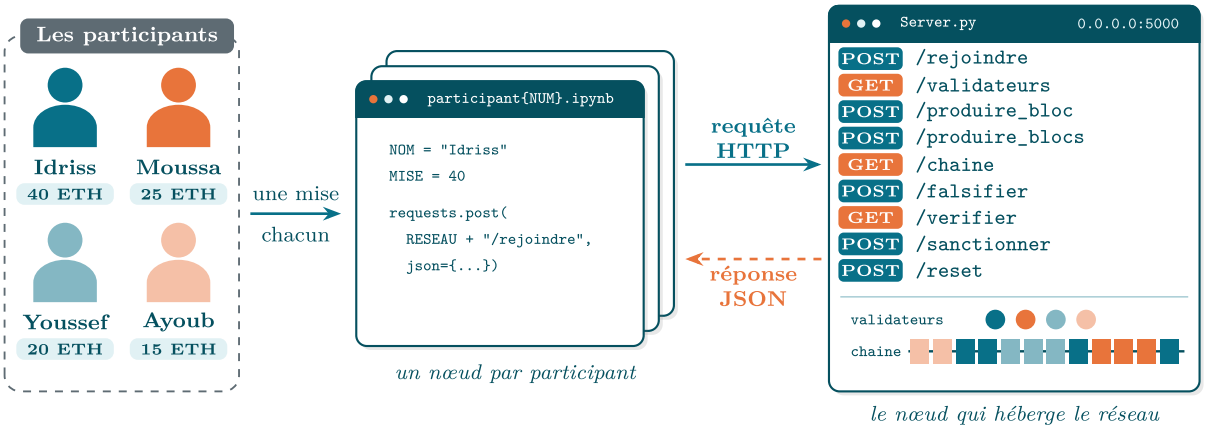
```
pip install flask requests
```

Deux fichiers vous sont fournis, avec les fonctions à écrire marquées `TODO` :

- `Server.py`, sur le nœud qui héberge le réseau.
- `participant{NUM}.ipynb`, chez chaque participant, `NUM` étant son numéro.

### Les étapes

1. `POST /rejoindre` : chacun engage sa mise. (R1)
2. `GET /validateurs` : lire les mises et les parts. (R2)
3. tirage à la main : vérifier la règle de sélection. (R2)
4. `POST /produire_bloc` : un bloc, avec son tirage. (R3, R4)
5. `POST /produire_blocs` : 14 blocs de plus. (R3)
6. `POST /produire_blocs` : 5 000 blocs, mesure des fréquences. (R2)
7. `POST /produire_blocs` avec `taux` : l'effet de la récompense. (R5)
8. `POST /falsifier` : un validateur signe un bloc qui ne lui revient pas. (R6)
9. `GET /verifier` : chacun recalcule et repère le bloc falsifié. (R6)
10. `POST /sanctionner` : le tricheur perd sa mise. (R7)




### Lancer le réseau

Sur le nœud qui héberge le réseau, dans un terminal laissé ouvert toute la séance :

```
python Server.py
```

```
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
```

Relever l'adresse IP de ce nœud avec `ipconfig` : c'est elle que les participants mettront dans `RESEAU`.

## 3 La simulation pas à pas

La première cellule du notebook, puis une étape par cellule.

```
import requests

RESEAU = "http://127.0.0.1:5000"   # sinon l'adresse IP du noeud qui heberge

print(requests.post(RESEAU + "/reset").json())
```

```
{'message': 'reseau vide, graine 2'}
```

La graine 2 fixe le générateur du serveur : les valeurs de ce sujet sont reproductibles. Un seul participant lance le `/reset`.

### Étape 1 : Rejoindre (R1)

Chaque participant exécute ces lignes avec son prénom et sa mise.

```
NOM = "Idriss"    # Moussa, Youssef, Ayoub...
MISE = 40         # 25, 20, 15

reponse = requests.post(RESEAU + "/rejoindre", json={"nom": NOM, "mise": MISE})
print(reponse.status_code, reponse.json())
```

Les quatre consoles, dans l'ordre des connexions :

```
200 {'mise_totale': 40, 'validateur': {'blocs': 0, 'exclu': False, 'mise': 40,
    'nom': 'Idriss'}}
200 {'mise_totale': 65, 'validateur': {'blocs': 0, 'exclu': False, 'mise': 25,
    'nom': 'Moussa'}}
200 {'mise_totale': 85, 'validateur': {'blocs': 0, 'exclu': False, 'mise': 20,
    'nom': 'Youssef'}}
200 {'mise_totale': 100, 'validateur': {'blocs': 0, 'exclu': False, 'mise': 15,
     'nom': 'Ayoub'}}
```

`mise_totale` augmente à chaque connexion : chacun voit le réseau grandir. Avec une mise sous le minimum, R1 refuse :

```
400 {'erreur': 'mise inferieure a 10'}
```

### Étape 2 : Les parts (R2)

Les validateurs sont demandés au réseau, seul détenteur de l'état.

```
validateurs = requests.get(RESEAU + "/validateurs").json()

total = 0
for v in validateurs:
    total = total + v["mise"]

print("mise totale :", total, "ETH")
for v in validateurs:
    print("{:<10} {:>5} ETH   {:5.1f} %".format(
        v["nom"], v["mise"], 100 * v["mise"] / total))
```

```
mise totale : 100 ETH
Idriss        40 ETH     40.0 %
Moussa        25 ETH     25.0 %
Youssef       20 ETH     20.0 %
Ayoub         15 ETH     15.0 %
```

### Étape 3 : Le tirage (R2)

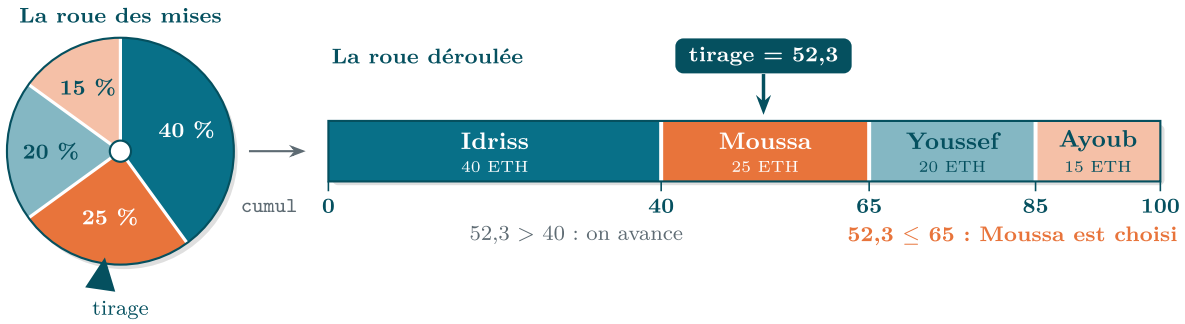
Chaque participant recopie la règle de sélection chez lui. C'est la fonction du serveur, et elle servira à le contrôler à l'étape 9.

```python
def choisir_proposant(mises, tirage):
    """R2, chez le participant."""
    cumul = 0
    for nom, mise in mises:
        cumul = cumul + mise
        if tirage <= cumul:
            return nom
    return mises[-1][0]


mises = [("Idriss", 40), ("Moussa", 25), ("Youssef", 20), ("Ayoub", 15)]
for t in [52.3, 12.0, 95.5]:
    print("tirage =", t, "-> proposant :", choisir_proposant(mises, t))
```

```
tirage = 52.3 -> proposant : Moussa
tirage = 12.0 -> proposant : Idriss
tirage = 95.5 -> proposant : Ayoub
```

Chaque validateur occupe sur le segment un intervalle égal à sa mise. Le tirage 52,3 dépasse le cumul 40 d'Idriss et s'arrête au cumul 65 : Moussa est désigné. Un intervalle plus long est atteint plus souvent, d'où $p_i = m_i / \sum_j m_j$.



### Étape 4 : Un bloc (R3, R4)

N'importe quel participant demande un bloc. Le réseau tire, désigne et renvoie le bloc.

```
bloc = requests.post(RESEAU + "/produire_bloc", json={}).json()
print(bloc)
print("recalcul chez moi :", choisir_proposant(bloc["mises"], bloc["tirage"]))
```

```
{'mises': [['Idriss', 40], ['Moussa', 25], ['Youssef', 20], ['Ayoub', 15]],
 'numero': 1, 'proposant': 'Ayoub', 'tirage': 95.6}
recalcul chez moi : Ayoub
```

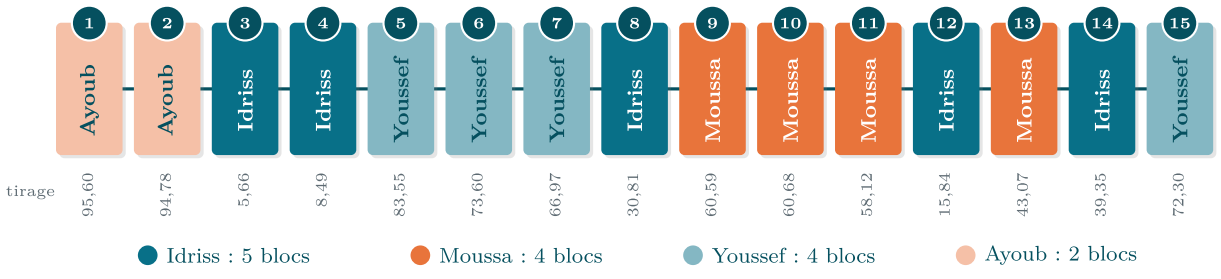
Le bloc transporte son tirage et les mises du moment (R4).

### Étape 5 : 14 blocs de plus (R3)

```
requests.post(RESEAU + "/produire_blocs", json={"n": 14})
chaine = requests.get(RESEAU + "/chaine").json()

print("Bloc  Proposant  Tirage")
for b in chaine:
    print("{:>4}  {:<9}  {:>6}".format(b["numero"], b["proposant"], b["tirage"]))
```

```
Bloc  Proposant  Tirage
   1  Ayoub       95.6
   2  Ayoub      94.78
   3  Idriss      5.66
   4  Idriss      8.49
   5  Youssef    83.55
   6  Youssef     73.6
   7  Youssef    66.97
   8  Idriss     30.81
   9  Moussa     60.59
  10  Moussa     60.68
  11  Moussa     58.12
  12  Idriss     15.84
  13  Moussa     43.07
  14  Idriss     39.35
  15  Youssef     72.3
```




Les validateurs sont tirés au moins une fois, mais Moussa et Youssef produisent autant de blocs alors que Moussa a misé davantage : sur 15 tirages les proportions ne sont pas atteintes.

### Étape 6 : Proportionnalité sur 5 000 blocs (R2)

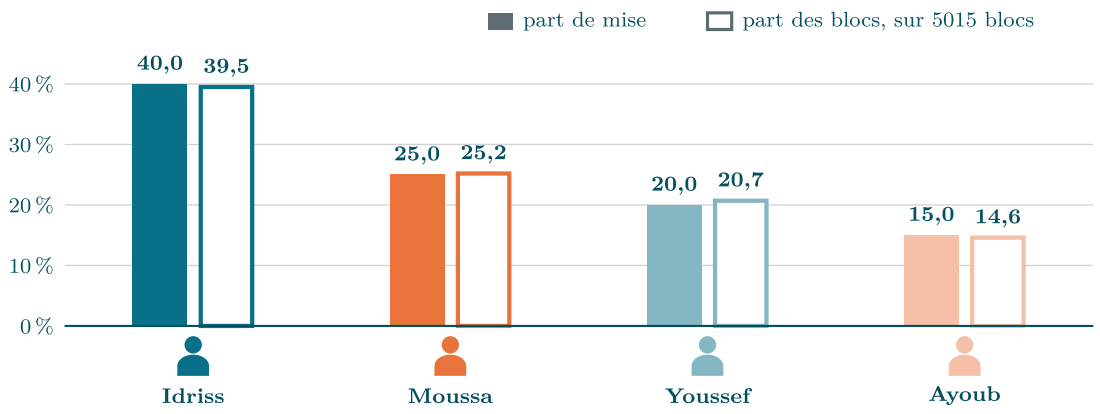
```
requests.post(RESEAU + "/produire_blocs", json={"n": 5000})

validateurs = requests.get(RESEAU + "/validateurs").json()
chaine = requests.get(RESEAU + "/chaine").json()
total = 0
for v in validateurs:
    total = total + v["mise"]

print("taille de la chaine :", len(chaine))
print("Validateur   Part de mise   Part des blocs")
for v in validateurs:
    part_mise = 100 * v["mise"] / total
    part_blocs = 100 * v["blocs"] / len(chaine)
    print("{:<10}   {:5.1f} %        {:5.1f} %   {}".format(
        v["nom"], part_mise, part_blocs, "#" * int(part_blocs / 2)))
```

```
taille de la chaine : 5015
Validateur   Part de mise   Part des blocs
Idriss         40.0 %         39.5 %   ####################
Moussa         25.0 %         25.2 %   ############
Youssef        20.0 %         20.7 %   ##########
Ayoub          15.0 %         14.6 %   #######
```

Les deux colonnes sont proches : $p_i = m_i / \sum_j m_j$ est vérifiée. L'écart le plus grand est celui de Youssef, 0,7 point.




Ce tableau sert aussi de contrôle : un validateur dont la part de blocs dépasse nettement sa part de mise a triché. C'est l'objet du test 4.

### Étape 7 : La récompense (R5)

Réseau vide, mêmes participants, R5 active : chaque bloc gagné augmente la mise de son proposant de 0,3 %.

```
requests.post(RESEAU + "/reset")
for nom, mise in [("Idriss", 40), ("Moussa", 25), ("Youssef", 20), ("Ayoub", 15)]:
    requests.post(RESEAU + "/rejoindre", json={"nom": nom, "mise": mise})

depart = {}
for v in requests.get(RESEAU + "/validateurs").json():
    depart[v["nom"]] = v["mise"]

requests.post(RESEAU + "/produire_blocs", json={"n": 1000, "taux": 0.003})

validateurs = requests.get(RESEAU + "/validateurs").json()
total_d = 0
total_f = 0
for v in validateurs:
    total_d = total_d + depart[v["nom"]]
    total_f = total_f + v["mise"]

print("Validateur   Mise depart   Mise fin   Part depart   Part fin   Blocs")
for v in validateurs:
    print("{:<10}   {:>9.1f}   {:>8.1f}   {:>9.1f} %   {:>6.1f} %   {:>5}".format(
        v["nom"], depart[v["nom"]], v["mise"],
        100 * depart[v["nom"]] / total_d, 100 * v["mise"] / total_f, v["blocs"]))
print("Reseau       {:>9.1f}   {:>8.1f}".format(total_d, total_f))
```

```
Validateur   Mise depart   Mise fin   Part depart   Part fin   Blocs
Idriss             40.0      175.2        40.0 %     62.9 %     493
Moussa             25.0       50.2        25.0 %     18.0 %     233
Youssef            20.0       31.3        20.0 %     11.2 %     149
Ayoub              15.0       21.8        15.0 %      7.8 %     125
Reseau            100.0      278.5
```

**Personne ne perd d'ETH.** R5 n'enlève rien à personne : les 4 mises augmentent. Moussa passe de 25 à 50,2 ETH, il a doublé sa mise, et sa part tombe pourtant de 25 % à 18 %.

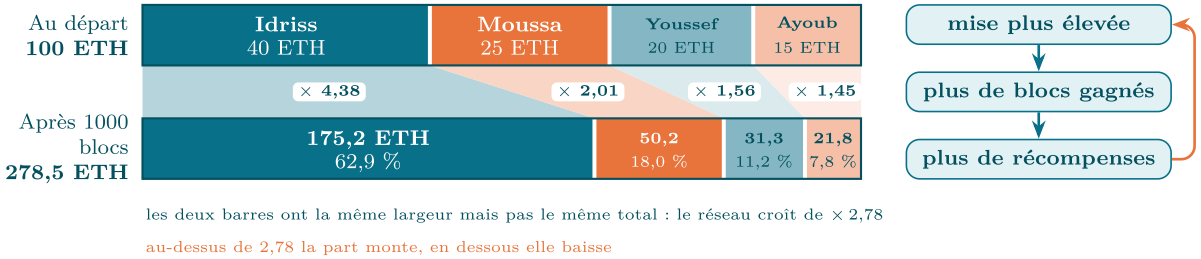
```
print("Validateur   Mise x    Part depart -> Part fin")
for v in validateurs:
    print("{:<10}   {:>5.2f}      {:>5.1f} % -> {:>5.1f} %".format(
        v["nom"], v["mise"] / depart[v["nom"]],
        100 * depart[v["nom"]] / total_d, 100 * v["mise"] / total_f))
print("Reseau       {:>5.2f}".format(total_f / total_d))
```

```
Validateur   Mise x    Part depart -> Part fin
Idriss         4.38        40.0 % ->  62.9 %
Moussa         2.01        25.0 % ->  18.0 %
Youssef        1.56        20.0 % ->  11.2 %
Ayoub          1.45        15.0 % ->   7.8 %
Reseau         2.78
```

Le réseau croît de ×2,78. Idriss est le seul au-dessus avec ×4,38 : sa part monte. Les 3 autres sont en dessous, leur mise augmente moins vite que le total, donc leur part baisse. Aucune règle ne les sanctionne, c'est une conséquence arithmétique de R5.

Idriss gagne 493 blocs sur 1000, soit 49,3 % des blocs pour 40 % de la mise au départ. Chaque bloc gagné augmente sa mise, donc sa chance de gagner le suivant : l'avance initiale s'amplifie

d'elle-même. C'est le risque de centralisation des protocoles où la récompense est proportionnelle à la mise.




## Étape 8 : Falsifier (R6)

Ayoub écrit un bloc à son nom alors que le tirage désigne quelqu'un d'autre. Réseau neuf, 5 blocs honnêtes, le bloc falsifié, puis 3 blocs honnêtes.

```
requests.post(RESEAU + "/reset")
for nom, mise in [("Idriss", 40), ("Moussa", 25), ("Youssef", 20), ("Ayoub", 15)]:
    requests.post(RESEAU + "/rejoindre", json={"nom": nom, "mise": mise})

requests.post(RESEAU + "/produire_blocs", json={"n": 5})
faux = requests.post(RESEAU + "/falsifier", json={"nom": "Ayoub"}).json()
requests.post(RESEAU + "/produire_blocs", json={"n": 3})

print("bloc", faux["numero"], "signe par", faux["proposant"],
      "| tirage", faux["tirage"])
print("recalcul :", choisir_proposant(faux["mises"], faux["tirage"]))
```

```
bloc 6 signe par Ayoub | tirage 73.6
recalcul : Youssef
```

Rien ne distingue le bloc 6 à l'œil : numéro, proposant et tirage, comme les autres.

## Étape 9 : Détecter (R6)

La fraude est détectable parce que R4 a été respectée. Chaque participant télécharge la chaîne et rejoue R2 sur chaque bloc, sans rien demander au réseau.

```
def verifier_bloc(bloc):
    """R6 : le nom inscrit doit etre celui que le tirage designe."""
    return bloc["proposant"] == choisir_proposant(bloc["mises"], bloc["tirage"])

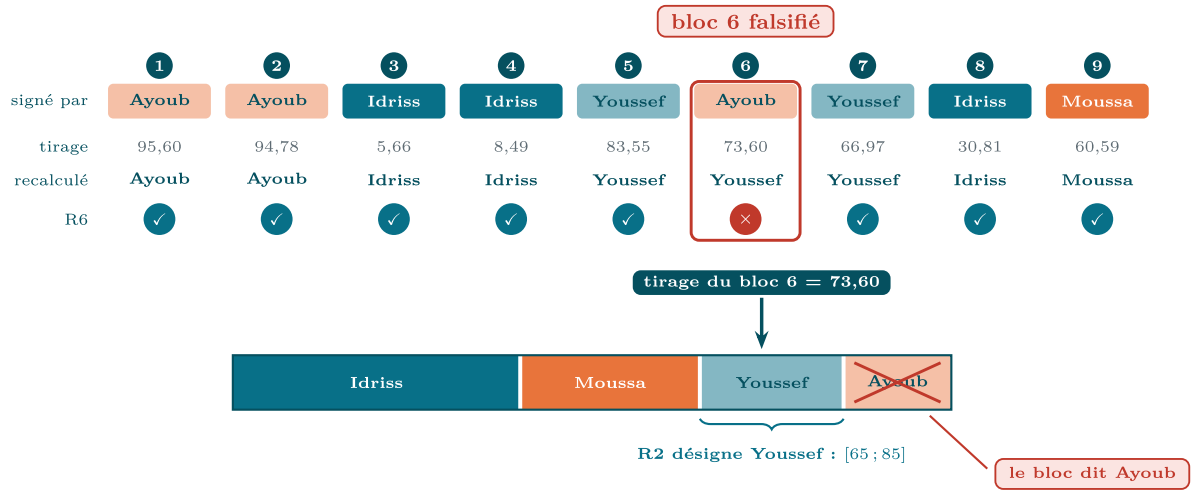

chaine = requests.get(RESEAU + "/chaine").json()

print("Bloc  Signe par  Tirage  Recalcul   Verdict")
for b in chaine:
    print("{:>4}  {:<9}  {:>6}  {:<9}  {}".format(
        b["numero"], b["proposant"], b["tirage"],
        choisir_proposant(b["mises"], b["tirage"]),
        "ok" if verifier_bloc(b) else "FALSIFIE"))

rep = requests.get(RESEAU + "/verifier").json()
print("reponse du reseau :", rep["blocs_falsifies"])
```

```
Bloc  Signe par  Tirage  Recalcul   Verdict
   1  Ayoub       95.6  Ayoub      ok
   2  Ayoub      94.78  Ayoub      ok
   3  Idriss      5.66  Idriss     ok
   4  Idriss      8.49  Idriss     ok
   5  Youssef    83.55  Youssef    ok
   6  Ayoub       73.6  Youssef    FALSIFIE
   7  Youssef    66.97  Youssef    ok
   8  Idriss     30.81  Idriss     ok
   9  Moussa     60.59  Moussa     ok
reponse du reseau : [6]
```

Un seul bloc est rejeté. Les blocs 1 et 2, produits par Ayoub qui n'a que 15 % de la mise, restent valides : avoir peu misé n'est pas une faute, les tirages 95,60 et 94,78 le désignent réellement.




## Étape 10 : Sanctionner (R7)

La détection n'a d'effet que si elle coûte quelque chose. R7 confisque la moitié de la mise et exclut le validateur, que `etat_des_mises` ignore désormais.

```python
for numero in requests.get(RESEAU + "/verifier").json()["blocs_falsifies"]:
    coupable = chaine[numero - 1]["proposant"]
    r = requests.post(RESEAU + "/sanctionner",
                      json={"nom": coupable, "fraction": 0.5}).json()
    print(coupable, ":", r)

total = 0
for v in requests.get(RESEAU + "/validateurs").json():
    if not v["exclu"]:
        total = total + v["mise"]
print("mise totale :", total, "ETH")
```

```
Ayoub : {'exclu': True, 'mise_restante': 7.5, 'perte': 7.5}
mise totale : 85 ETH
```

La mise totale passe de 100 à 85 ETH. Les parts des 3 validateurs restants augmentent donc mécaniquement, sans qu'ils aient rien fait : c'est le calcul de l'étape 7, dans l'autre sens.

# 4 Testez le réseau

Chaque test change une valeur et relance les étapes indiquées. En gardant la graine 2, vous obtenez exactement ces sorties.

1. **La taille de l'échantillon.** À l'étape 6, remplacez `5000` par `500`, puis relancez les étapes 1 à 6. Recommencez avec `100000`.

   Avec 500 blocs :

   ```
   Validateur   Part de mise   Part des blocs
   Idriss         40.0 %         39.4 %   ####################
   Moussa         25.0 %         24.6 %   ############
   Youssef        20.0 %         20.0 %   ##########
   Ayoub          15.0 %         16.0 %   ########
   ```

   Avec 100 000 blocs :

   ```
   Validateur   Part de mise   Part des blocs
   Idriss         40.0 %         39.9 %   ####################
   Moussa         25.0 %         25.0 %   #############
   Youssef        20.0 %         20.1 %   ##########
   Ayoub          15.0 %         15.0 %   #######
   ```

   L'écart maximal passe de 1,0 point à 0,1 point. R2 n'est exacte qu'à la limite.

2. **L'arrivée de 3 validateurs.** Remettez `5000` à l'étape 6. Aymen rejoint avec 100 ETH, Timan et Meryem avec 50 ETH chacun, puis relancez les étapes 1 à 6.

   ```
   Validateur   Part de mise   Part des blocs
   Idriss         13.3 %         13.2 %   ######
   Moussa          8.3 %          8.2 %   ####
   Youssef         6.7 %          6.8 %   ###
   Ayoub           5.0 %          4.9 %   ##
   Aymen          33.3 %         33.4 %   ################
   Timan          16.7 %         17.3 %   ########
   Meryem         16.7 %         16.3 %   ########
   ```

   Aymen détient un tiers de la mise et produit un tiers des blocs. Timan et Meryem, à mise égale, produisent presque autant de blocs. Les 4 premiers n'ont pas touché à leur mise et voient leur part divisée par 3 : le dénominateur de l'étape 7 a bougé.

3. **Une récompense plus forte.** Revenez aux 4 participants et portez `taux` de `0.003` à `0.01` à l'étape 7.

   ```
   Validateur   Mise depart   Mise fin   Part depart   Part fin   Blocs
   Idriss            40.0    93914.3        40.0 %     99.9 %     780
   Moussa            25.0       69.0        25.0 %      0.1 %     102
   Youssef           20.0       37.1        20.0 %      0.0 %      62
   Ayoub             15.0       26.2        15.0 %      0.0 %      56
   Reseau           100.0    94046.5
   ```

   Idriss finit avec 99,9 % de la mise. Moussa a presque triplé la sienne, de 25 à 69 ETH, mais le réseau a été multiplié par 940 : la lecture en pourcentage le rend invisible.

4. **Un tricheur plus habile.** Ayoub ne falsifie plus le nom : il choisit un tirage qui tombe dans son propre intervalle et l'envoie au réseau. Le bloc reste cohérent, donc le contrôle de R6 ne voit rien.

```python
import random

requests.post(RESEAU + "/reset")
for nom, mise in [("Idriss", 40), ("Moussa", 25), ("Youssef", 20), ("Ayoub", 15)]:
    requests.post(RESEAU + "/rejoindre", json={"nom": nom, "mise": mise})

random.seed(2)
mises = [("Idriss", 40), ("Moussa", 25), ("Youssef", 20), ("Ayoub", 15)]
debut = 85    # l'intervalle d'Ayoub
fin = 100

for i in range(400):
    if i % 4 == 0:
        tirage = round(random.uniform(debut, fin), 2)
        requests.post(RESEAU + "/falsifier", json={"nom": "Ayoub", "tirage":
    tirage})
    else:
        requests.post(RESEAU + "/produire_bloc", json={})

rep = requests.get(RESEAU + "/verifier").json()
print("blocs falsifies :", rep["blocs_falsifies"])

print("Validateur   Part de mise   Part des blocs   Verdict")
for f in rep["frequences"]:
    print("{:<10}   {:>8.1f} %      {:>8.1f} %   {}".format(
        f["nom"], f["part_mise"], f["part_blocs"],
        "SUSPECT" if f["suspect"] else "normal"))
```

```
blocs falsifies : []
Validateur   Part de mise   Part des blocs   Verdict
Idriss            40.0 %           29.2 %   normal
Moussa            25.0 %           19.8 %   normal
Youssef           20.0 %           14.0 %   normal
Ayoub             15.0 %           37.0 %   SUSPECT
```

Chaque bloc est cohérent avec son propre tirage, donc `blocs_falsifies` reste vide. La fraude se voit dans les fréquences : Ayoub produit 37,0 % des blocs avec 15 % de la mise. Les deux contrôles sont complémentaires, l'un attrape le bloc incohérent, l'autre le tirage truqué. Baissez la proportion de blocs truqués à 1 sur 20 et cherchez à partir de combien de blocs l'écart redevient indiscernable du hasard.

### Limite de la simulation

Ici le tirage est écrit par celui qui produit le bloc, et seules les fréquences dénoncent un tricheur patient. Un réseau réel supprime cette liberté : le tirage vient d'une fonction aléatoire vérifiable (VRF) et le bloc est signé. Chacun vérifie le tirage sans pouvoir le choisir.

# 5 Structure des fonctions

**participant{NUM}.ipynb**

```
import requests

RESEAU = "http://127.0.0.1:5000"   # ou l'adresse IP du noeud qui heberge
NUM = 1                            # votre numero
NOM = "Idriss"                     # votre prenom
MISE = 40                          # >= 10


def choisir_proposant(mises, tirage):
    """R2, recopiee chez le participant pour controler le reseau.
    Parametres : mises (liste de couples (nom, mise)), tirage (float)
    Retour     : str, le nom du proposant"""
    ...


def verifier_bloc(bloc):
    """R6 : controle un bloc telecharge, sans rien demander au reseau.
    Parametres : bloc (dict : numero, proposant, tirage, mises)
    Retour     : bool, True si le bloc est valide"""
    ...


def verifier_chaine(chaine):
    """R6 : applique verifier_bloc a toute la chaine.
    Parametres : chaine (liste de dict)
    Retour     : liste des numeros (int) des blocs falsifies"""
    ...


def parts(mises):
    """Part de chaque validateur dans la mise totale.
    Parametres : mises (liste de couples)
    Retour     : dict {nom (str): pourcentage (float)}"""
    ...


def afficher_parts(validateurs, chaine):
    """Tableau part de mise / part des blocs, avec barres de '#'.
    Parametres : validateurs (liste de dict), chaine (liste de dict)
    Retour     : aucun (affichage)"""
    ...
```

## Rendu

Déposer sur la plateforme du module `Server.py` et `participant{NUM}.ipynb` exécuté, sorties visibles.